
A Comparative Study of Density-Based and Generative Resampling Methods for Imbalanced Credit Card Fraud Detection

Shaun Campbell¹

Abstract

Credit card fraud resulted in approximately \$33 billion in global losses in 2024. A major challenge in developing automated detection models is the extreme class imbalance in financial fraud datasets. This research provides a comparative study of Kernel Density Estimation (KDE) and Generative Adversarial Networks (GANs) as advanced alternatives for synthetic data generation for credit card fraud data. A GAN pipeline that integrates density-based noise filtering and boundary refinement was implemented, and the results indicate that while density-based methods offer high statistical fidelity, the GAN-based pipeline can significantly improve F-1 and precision in gradient-based classifiers.

1. Introduction

Sophisticated machine learning and AI methods are widely used in fraud detection. However, credit card fraud still led to global losses of \$33 billion in 2024 ([The Nilson Report, 2024](#)). A primary technical barrier in developing fraud detection models is the extreme class imbalance, with fraudulent transactions making up only around 0.17% of total data ([Pozzolo et al., 2015](#)).

Traditionally, random undersampling, random oversampling, and Synthetic Minority Over-sampling Technique (SMOTE) have been used to address class imbalance ([Alamri & Ykhlef, 2022](#)). However, these methods often fail to capture the actual underlying data distributions, introduce noise, and cause models to overgeneralize the minority class.

This research investigates advanced statistical and generative resampling methods, specifically Kernel Density Estimation (KDE) and Generative Adversarial Networks (GANs), in the context of credit card fraud. The goal is

to provide a comprehensive comparison of these methods in terms of the structure of the synthetic data they produce and their performance impact on classification models. The results demonstrate that KDE is capable of producing synthetic data more closely aligned with empirical distributions, which led to a slight improvement in F-1 and recall with a random forest model. The implementation of a DB-CTGAN-ENN pipeline that integrates density-based noise filtering and boundary cleaning with a GAN generator was able to successfully capture a joint probability distribution of fraud samples. This led to improved F-1 scores in gradient-based classifiers over SMOTE and KDE.

2. Related Work

Traditionally, the challenge of imbalanced data in fraud detection has been addressed with heuristic sampling. SMOTE ([Chawla et al., 2002](#)) remains a prominent method, using linear interpolation to generate synthetic data between empirical minority samples. Over 20 years since its inception, SMOTE is still considered a standard benchmark for imbalanced data ([Fernández et al., 2018](#)) and has been used widely in other fraud detection studies (e.g., [Varmedja et al., 2019](#); [Dornadula & Geetha, 2019](#)). Several variations of SMOTE, such as SMOTE-Tomek ([Batista et al., 2003](#)), SMOTE-ENN ([Batista et al., 2004](#)), Borderline-SMOTE ([Han et al., 2005](#)), and ADASYN ([He et al., 2008](#)) have been proposed to improve upon the basic methodology. However, SMOTE methods may not match the underlying distribution of the minority class ([Elreedy et al., 2024](#)) and can overgeneralize the minority class region ([Maciejewski & Stefanowski, 2011](#)).

To overcome the limitations of linear interpolation, density-based methods have been proposed. Such approaches aim to model the underlying distribution of the minority class more comprehensively. Previous studies have explored the use of KDE to generate samples from an estimated probability density function (PDF). This approach has been found to perform well and be statistically sound in several domains (e.g., [Kamalov, 2020](#); [Taccaliti & Aguilar-Ruiz, 2025](#)). However, using KDE to model the joint probability distribution of the minority class is difficult in high dimensions, especially with few minority samples, as the number of data points must

¹Department of Computer Science, Marquette University, Milwaukee, WI, USA. Correspondence to: Shaun Campbell <shaun.campbell@marquette.edu>.

scale exponentially with the number of dimensions to get a good approximation (Biroli & Mézard, 2026). KDE must then rely on dimensionality reduction or modeling each feature individually. The former introduces information loss, and the latter causes loss of correlations.

Recent advances in generative models have led to generative adversarial networks (GANs) designed specifically to generate tabular data. The Conditional Tabular GAN (CTGAN) (Xu et al., 2019) uses mode-specific normalization and a conditional generator to handle non-Gaussian and multimodal data. However, synthetic data generated by GANs frequently suffers from high class overlap. A growing body of research has focused on addressing this through hybrid approaches. GBHS (Zhu et al., 2022), HS-CGK (Zhao et al., 2024), and GANHS (Xie et al., 2025) are examples of such work. These hybrid methods use techniques such as pre-processing to filter noise prior to training the GAN and post-processing techniques to handle the class overlap of generated samples.

Although several studies have compared the effects of sampling techniques on model performance for fraud detection (e.g., Praveen Mahesh et al., 2022; Alamri & Ykhlef, 2022; Zhao et al., 2024), there is limited comparative research focused on the structural differences between probabilistic (KDE) and generative (GAN) sampling methods for credit card fraud data. This work addresses this gap by providing statistical comparisons, qualitative analysis via Uniform Manifold Approximation and Projection (UMAP), and comparative performance evaluation across several classifiers.

3. Methodology

3.1. Dataset and Preprocessing

The dataset used in this analysis was the Credit Card Fraud Detection dataset from Kaggle (Pozzolo et al., 2015). It contains 284,807 European credit card transactions with 492 labeled fraud cases. It has 28 PCA-transformed features plus features for time and amount.

Standard scaling pipelines were utilized throughout the analysis for every variable to remove feature scale as a factor. A 75-25 train-test split was performed to reserve 25% of the data for the final evaluation. Due to the extreme class imbalance, the split was stratified by class to ensure that the train and test sets contained the same ratio of fraud samples.

3.2. Synthetic Data Quality Metrics

Several metrics for synthetic data quality were used to compare the various sampling methods. **Class Overlap Rate (OLR)** estimates how many synthetic samples overlap with the majority class. The metric designates a synthetic minority sample as overlapping if its nearest neighbor in Euclidean

space is a member of the majority class. The **Kolmogorov-Smirnov Test (KS)** is a statistical test used to determine the likelihood that the synthetic data and the original data came from the same distribution. In this analysis, the average KS statistic across individual features is reported. **Maximum Mean Discrepancy (MMD)** compares the kernel mean embeddings of the synthetic distribution and original distribution in reproducing kernel Hilbert space. **Wasserstein Distance (WD)** measures the cost of converting the synthetic distribution into the original distribution. In this analysis, the average WD across individual features is reported.

3.3. Resampling Techniques

3.3.1. TRADITIONAL METHODS: RUS AND SMOTE

Random undersampling (RUS) served as a baseline resampling method as it is very simple and straightforward. RUS randomly removes samples of the majority class (non-fraudulent transactions) until the target class ratio is reached. SMOTE served as the baseline for synthetic data generation. SMOTE generates synthetic data by interpolating between minority (fraud) data samples to oversample to the target class ratio.

Imbalanced-learn (Lemaître et al., 2017) was used for RUS and SMOTE, and the BaseSampler class was used to create the proceeding samplers.

3.3.2. KERNEL DENSITY ESTIMATION

KDE is a method used to estimate the PDF of a random variable and is especially useful for unknown or non-normal distributions. It works by placing a kernel function over each data point and then summing them to create a smooth density estimate. For a univariate distribution, the density estimate at a given point y is estimated as:

$$\hat{f}_h(y) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{y - x_i}{h}\right) \quad (1)$$

Where n is the number of data points, x_i are the data points, K is the kernel function, and h is a smoothing parameter called the bandwidth that balances the tradeoff between bias and variance. Synthetic data can then be generated by sampling from the estimated PDF. The KernelDensity module from scikit-learn (Pedregosa et al., 2011) was used for implementation in this analysis.

The normality of the features in the dataset was first assessed to justify using KDE over Gaussian sampling. The features all have high skewness and kurtosis and fail the D'Agostino and Pearson's normality test, so using KDE is justified.

First, a multivariate KDE sampler was explored to oversample using a joint probability density function of all the

features for fraud cases using a Gaussian kernel. As expected, multivariate KDE was not ideal due to the high-dimensionality and few fraud cases. The resulting densities were overly wide and smooth compared to the original densities because of the high bandwidths required.

A univariate KDE sampler was created instead to oversample each variable independently from its estimated PDF. The sampler works by iterating through the features and modeling each density individually with a Gaussian kernel, using grid search to tune the bandwidth for optimal log-likelihood. Values are then sampled from each feature’s PDF to compile synthetic observations.

A typical shortcoming of univariate KDE is that it does not capture correlations between variables. However, since all but two of the features are PCA transformed, little correlation exists to begin with since PCA removes correlation (a correlation heatmap is included in Appendix A). Because of this, a univariate approach is adequate for this dataset.

3.3.3. GENERATIVE ADVERSARIAL NETWORKS

Using GANs to generate synthetic data is an emerging method to handle class imbalance. GANs work by training two neural networks, a generator and discriminator, that compete against each other. The resulting generator can then be used generate realistic synthetic data. In this analysis, GAN-based sampling was implemented using the CTGAN library (Xu et al., 2019) which was specifically designed to generate synthetic tabular data.

Bayesian optimization was performed using Optuna (Akiba et al., 2019) to tune the layer dimensions, learning rates, and packing number. 20 trials were performed with 1,000 epochs in each and the convergence plot is included in Appendix B. The resulting GAN did not perform as well as SMOTE and KDE in terms of synthetic data metrics, with around 25% of the synthetic data overlapping with the majority class.

Such performance is common with GAN resampling and has been addressed in previous research by filtering noise prior to training and undersampling to remove overlapping samples from the generated data. Both of these techniques were implemented in this analysis, and the resulting method is very similar to that in (Zhao et al., 2024). The primary difference between the method proposed by Zhao et al. and the one implemented here is that Zhao et al. use an edited-nearest neighbors (ENN) approach to filter *majority* samples from the overlap region, while the method here only filters *minority* samples generated by the GAN.

This method, referred to as DB-CTGAN-ENN in this analysis, first uses the DBSCAN algorithm to filter noisy samples from the fraud training data. The heuristic elbow method was used to select 13 as the epsilon parameter and 10 train-

ing fraud samples were removed (k-distance graph included in Appendix C). Then Bayesian optimization was performed to find the optimal parameters for the GAN.

To oversample, the DB-CTGAN-ENN sampler generates more samples than required to meet the target sampling ratio. It then removes generated samples if a k-nearest-neighbor classifier, trained on the original training data, classifies them as majority samples. If more samples are needed after the ENN filter, the GAN repeats the process to add more samples. It then returns the number of synthetic samples required for the target sampling ratio. The number of neighbors (k) and the Minkowski distance power (p) were tuned by selecting the combination that minimized MMD (k=10, p=2). This method performed better than the original GAN in terms of synthetic data metrics, suggesting that the DBSCAN and ENN steps helped refine the resulting synthetic data distributions closer to the originals.

4. Experimental Setup

4.1. Classifier Selection

To assess the efficacy of the resampling methods for fraud data, three classifiers were chosen for evaluation: logistic regression (LR), multi-layered perceptron (MLP), and random forest (RF). LR was selected as a simple, linear model. MLP was used to evaluate a deep-learning model. RF was included due to its high performance in other fraud detection studies (Varmedja et al., 2019).

4.2. Hyperparameter Tuning

It was found that with a 50/50 sampling ratio, all models overpredicted fraud resulting in very high recalls but very low precisions. To address this, the sampling ratio was tuned as a hyperparameter. The training data was used to test several sampling ratios using SMOTE and a decision tree classifier. SMOTE was used instead of RUS because with the latter, higher sampling ratios would bias towards degraded performance due to more data being thrown out. A decision tree classifier was used because it is a simple nonlinear method that should translate to RF and MLP. It was found that a 0.1 sampling ratio resulted in the best balance between precision and recall.

The hyperparameters of each model were then tuned with the 0.1 sampling ratio. Random undersampling was used both for efficiency and to avoid introducing bias towards synthetic data. Grid search with repeated stratified k-fold was used for LR and RF. For MLP, the optimal parameters found by (Varmedja et al., 2019) were used for hidden layer sizes, activation function, and solver, and Bayesian search was used to tune alpha, initial learning rate, and batch size. Scikit-learn (Pedregosa et al., 2011) was used to implement each model.

4.3. Performance Evaluation Metrics

Accuracy is a misleading metric in the presence of extreme class imbalance, so several other metrics were used instead. Recall, precision, and F-1 score were chosen to evaluate the models' ability to identify fraud accurately without being influenced by the high prevalence of true negatives. Area Under the Precision-Recall Curve (AUPRC) was used instead of the standard AUROC because the former is a more robust measure for a classifier's ability to distinguish minority classes in highly imbalanced data.

5. Results

5.1. Synthetic Data Fidelity

The quality metrics for the synthetic data sampling methods are shown in Table 1. Density plots of the synthetic fraud samples from each method, together with the empirical (training) densities, are shown in Figure 1 for three select features.

Table 1. Synthetic data quality metrics for the various sampling methods.

METHOD	OLR	KS	MMD	WD
SMOTE	0.0389	0.0410	0.0306	0.1401
UNIVARIATE KDE	0.0169	0.0455	0.0218	0.1037
CTGAN	0.3523	0.1446	0.3042	0.9005
DB-CTGAN-ENN	0.0629	0.1183	0.2519	0.6887

SMOTE had the best mean Kolmogorov-Smirnov test statistic, which is intuitive because SMOTE interpolates points between existing points. Notably, SMOTE still had an overlap rate of 3.61%, which means that despite only generating points between existing points, some of the resulting samples were closest to the majority class. It can be seen from the densities in Figure 1 that SMOTE tends to exaggerate local modalities in the distributions.

Univariate KDE had the lowest overlap rate, MMD, and mean Wasserstein distance. This implies that the individual feature distributions of the KDE-generated fraud cases closely mirrored the empirical distributions. KDE excels at modeling the PDF of each feature, and the resulting synthetic data distributions reflect this, demonstrated in Figure 1. KDE still exaggerates some features of the original distributions, but is generally closer than SMOTE.

Implementing DBSCAN and ENN improved the GAN metrics. The overlap rate obviously decreased markedly due to the implementation of ENN to filter overlapping samples. The heuristic and statistical methods (SMOTE and KDE) had better quality metrics, but this just suggests that those methods are better at mimicking the original distributions. In contrast, the results for DB-CTGAN-ENN suggest that

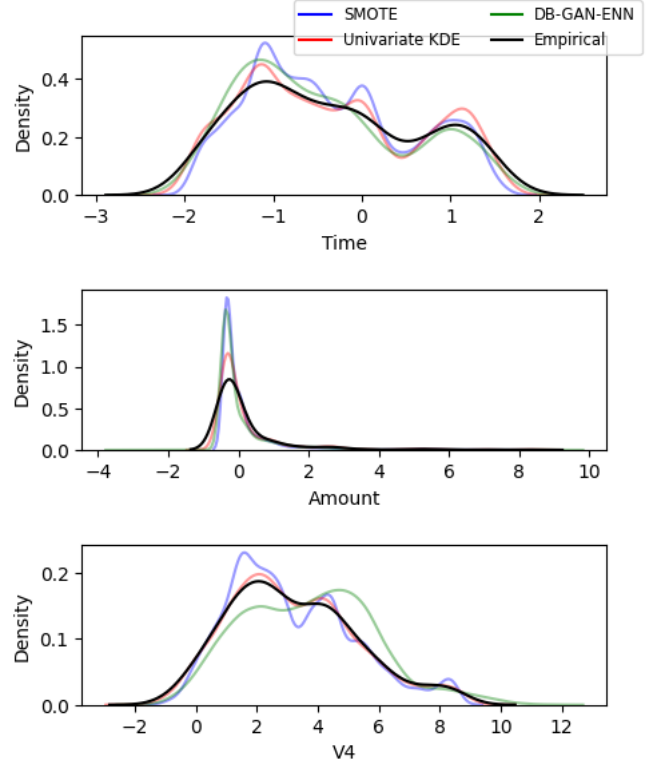


Figure 1. Density plots of empirical and synthetic fraud samples for each method on select features.

the model is exploring a broader but still plausible feature space. This is evidenced by Figure 1, as the synthetic densities generally align with the empirical distributions despite having less statistical similarity.

UMAP (McInnes et al., 2018) was used to project synthetic and empirical fraud samples into two-dimensional space, and the results for each method are shown in Figure 2. The synthetic SMOTE samples have clearly evident linear patterns that connect the original fraud samples as a result of linear interpolation.

The synthetic samples generated with KDE show a seemingly randomized "spray" near and between the original fraud samples. This can be attributed to the use of *univariate* KDE, which treats every feature's probability distribution as independent, resulting in synthetic samples near the original samples but not direct linear interpolations like in SMOTE. The space between the original fraud samples is filled more uniformly using KDE than it is with SMOTE.

The CTGAN synthetic fraud samples form a dense cloud that encapsulates the original fraud samples. This suggests that the GAN successfully learned a joint probability density of fraud cases across all the features. The noise removal and decision boundary refinement with the DB-CTGAN-ENN method slightly thinned out the cloud, and the apparent out-

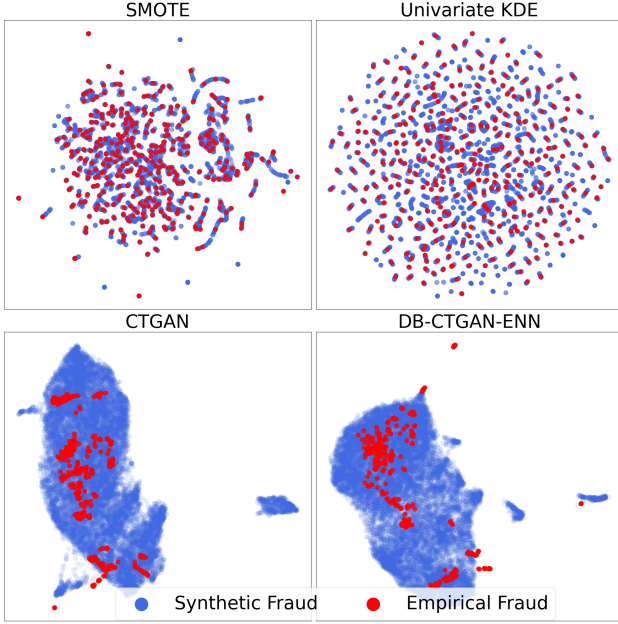


Figure 2. UMAP projections of empirical and synthetic fraud samples for each method.

lier regions have less of an effect on the synthetic manifold.

5.2. Classifier Performance

The results of the experiment on the test set are shown in Table 2, and the results on the train set are in Appendix D. DB-CTGAN-ENN was the best resampling method to help the gradient-based models of logistic regression and MLP identify fraud. SMOTE and univariate KDE were the best for random forest, which was the best-performing model overall.

The univariate KDE method demonstrated performance similar to SMOTE, with slight gains in F-1, AUPRC, and precision in MLP and slight declines in performance in logistic regression. In random forest, univariate KDE provided a slight boost to recall and F-1 score over SMOTE.

DB-CTGAN-ENN method consistently outperformed the base CTGAN, with higher precision, F-1, and AUPRC scores in all models. Significant increases in precision were accompanied by only modest drops in recall, which is attributable to the noise removal and refinement of the decision boundary. Although this led to missing slightly more fraud cases (recall), it helped the models to be much more accurate in their fraud predictions (precision).

5.3. Discussion

The experimental results reveal the differences between traditional, density-based, and generative resampling methods for credit card fraud data. Adding legitimate transactions to

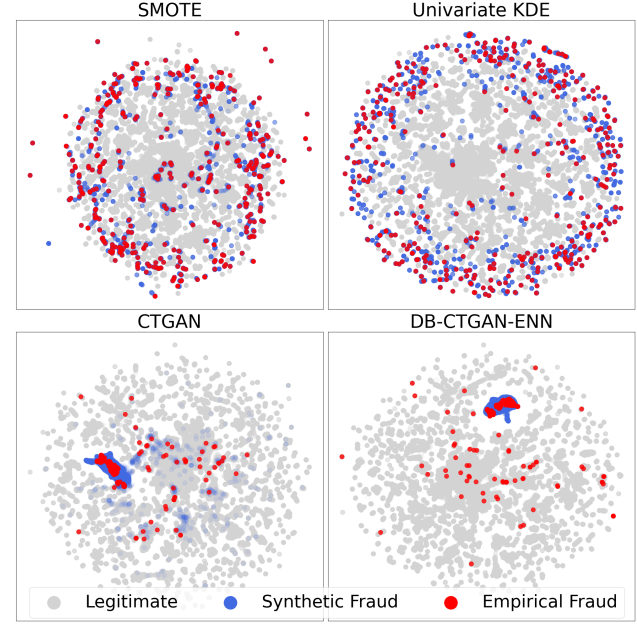


Figure 3. UMAP projections of legitimate transactions, empirical fraud, and synthetic fraud samples for each method.

the UMAP projections, shown in Figure 3, showcases the distinctions in how the resampling methods affect the fraud manifold within the feature space.

The univariate KDE method functioned similarly to SMOTE by generating samples located near the original fraud observations and dispersed across a broad region, though KDE appears to provide a slightly broader coverage of the fraud region versus SMOTE. It accomplished this by generating samples with statistically likely feature values instead of the linear interpolation used in SMOTE, and while the two methods use different strategies, the resulting performance in the models were very closely aligned. However, the combination of univariate KDE and random forest produced the best F-1 score across all models and methods.

The DB-CTGAN-ENN method successfully learned a continuous joint probability density for fraud, evident in Figure 2 and also seen in Figure 3. The prevalent overlap in the CTGAN method can be seen as synthetic samples are scattered within the legitimate region. The combination of noise filtering and overlap reduction successfully confined the synthetic samples to an isolated fraud region. This explains the slight degradation in recall and the significant improvement in precision in DB-CTGAN-ENN versus CTGAN, as it led the models to focus on the clear fraud region, improving their ability to identify fraud there, but ignore the minority of cases outside the region.

The fact that DB-CTGAN-ENN improved performance in LR and MLP but did not in RF is explainable with Figure 3. The DB-CTGAN-ENN method generated samples that

Table 2. Classifier performance metrics on the test set

CLASSIFIER	SAMPLING METHOD	RECALL	PRECISION	F-1	AUPRC
LOGISTIC REGRESSION	RUS	0.8293	0.6892	0.7528	0.7091
	SMOTE	0.8374	0.6242	0.7153	0.7167
	UNIVARIATE KDE	0.8374	0.5886	0.6913	0.7117
	CTGAN	0.8049	0.5562	0.6578	0.6702
	DB-CTGAN-ENN	0.7805	0.8205	0.8000	0.7111
MLP	RUS	0.8455	0.4464	0.5843	0.7146
	SMOTE	0.8455	0.6303	0.7222	0.7207
	UNIVARIATE KDE	0.8699	0.6185	0.7230	0.7388
	CTGAN	0.8130	0.7692	0.7905	0.7129
	DB-CTGAN-ENN	0.7886	0.8220	0.8050	0.7130
RANDOM FOREST	RUS	0.8455	0.6667	0.7455	0.7927
	SMOTE	0.8049	0.9000	0.8498	0.8600
	UNIVARIATE KDE	0.8130	0.8929	0.8511	0.8552
	CTGAN	0.8211	0.8080	0.8145	0.8340
	DB-CTGAN-ENN	0.8130	0.8621	0.8368	0.8370

established a clearly defined fraud region, which provided a stable training surface for the gradient-based methods to optimize their weights. Since random forest is naturally robust to noise and its decision boundary is partition-based, having a broad coverage of the general fraud regions, which was provided by SMOTE and KDE, proved more beneficial than having an isolated fraud surface provided by DB-CTGAN-ENN.

6. Conclusion

This research provides a comparative study on the structural and performative differences between probabilistic and generative sampling methods for credit card fraud data. Although univariate KDE is constrained by feature independence assumptions, it was demonstrated that KDE is capable of generating synthetic data more statistically similar to empirical distributions than SMOTE. This statistical advantage translated into an improved F-1 score and recall in the random forest model, a classifier frequently cited for high performance in fraud detection. Although the goal here was not to maximize F-1 score through meticulous hyperparameter tuning, this boost to F-1 and recall is worthy of further exploration. Increasing recall, even slightly, without a significant drop in precision would allow fraud detection models to catch more instances of fraud and reduce financial losses for credit card providers.

The UMAP projections revealed that GAN models are able to learn a continuous joint probability density for fraud instances. The implementation of pre-processing noise removal and post-sampling boundary refinement was also shown to be a useful and necessary component of GAN-based sampling. This refinement allowed gradient-based models, namely logistic regression and MLP, to leverage the smooth and isolated training manifold to improve per-

formance over SMOTE. The lack of improvement in the random forest model suggests that GAN-based sampling is not a universal solution but rather a model-dependent optimization tool that can excel in high-dimensional imbalanced data. Nonetheless, as fraud patterns become increasingly sophisticated, the use of generative models that can continuously adapt to changing fraud manifolds offers a powerful tool for automated fraud detection.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- Alamri, M. and Ykhlef, M. Survey of credit card anomaly and fraud detection using sampling techniques. *Electronics*, 11(23):4003, 2022.
- Batista, G. E., Bazzan, A. L., Monard, M. C., et al. Balancing training data for automated annotation of keywords: a case study. *Wob*, 3:10–18, 2003.
- Batista, G. E., Prati, R. C., and Monard, M. C. A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD explorations newsletter*, 6(1):20–29, 2004.
- Biroli, G. and Mézard, M. Kernel density estimators in large dimensions. *SIAM Journal on Mathematics of Data Science*, 8(1):46–76, 2026.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002.

- Dornadula, V. N. and Geetha, S. Credit card fraud detection using machine learning algorithms. *Procedia computer science*, 165:631–641, 2019.
- Elreedy, D., Atiya, A. F., and Kamalov, F. A theoretical distribution analysis of synthetic minority oversampling technique (smote) for imbalanced learning. *Machine Learning*, 113(7):4903–4923, 2024.
- Fernández, A., Garcia, S., Herrera, F., and Chawla, N. V. Smote for learning from imbalanced data: progress and challenges, marking the 15-year anniversary. *Journal of artificial intelligence research*, 61:863–905, 2018.
- Han, H., Wang, W.-Y., and Mao, B.-H. Borderline-smote: a new over-sampling method in imbalanced data sets learning. In *International conference on intelligent computing*, pp. 878–887. Springer, 2005.
- He, H., Bai, Y., Garcia, E. A., and Li, S. Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*, pp. 1322–1328. Ieee, 2008.
- Kamalov, F. Kernel density estimation based sampling for imbalanced class distribution. *Information Sciences*, 512: 1192–1201, 2020.
- Lemaître, G., Nogueira, F., and Aridas, C. K. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017. URL <http://jmlr.org/papers/v18/16-365.html>.
- Maciejewski, T. and Stefanowski, J. Local neighbourhood extension of smote for mining imbalanced data. In *2011 IEEE symposium on computational intelligence and data mining (CIDM)*, pp. 104–111. IEEE, 2011.
- McInnes, L., Healy, J., and Melville, J. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Pozzolo, A. D., Caelen, O., Johnson, R. A., and Bontempi, G. Credit card fraud detection dataset. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>, 2015. Accessed: 2026-04-11.
- Praveen Mahesh, K., Ashar Afrouz, S., and Shaju Areecal, A. Detection of fraudulent credit card transactions: A comparative analysis of data sampling and classification techniques. In *Journal of Physics: Conference Series*, volume 2161, pp. 012072. IOP Publishing, 2022.
- Taccaliti, E. and Aguilar-Ruiz, J. S. Improving classification on imbalanced genomic data via kde-based synthetic sampling. *BioData Mining*, 18(1):60, 2025.
- The Nilson Report. Global card fraud losses projected to reach \$33 billion by 2026. <https://finance.yahoo.com/news/global-card-fraud-losses-33-170800960.html>, 2024. Accessed: 2026-04-11.
- Varmedja, D., Karanovic, M., Sladojevic, S., Arsenovic, M., and Anderla, A. Credit card fraud detection-machine learning methods. In *2019 18th International Symposium Infoteh-Jahorina (Infoteh)*, pp. 1–5. IEEE, 2019.
- Xie, Y., Shan, J., Wei, L., Yao, J., and Zhou, M. Gan-based hybrid sampling method for transaction fraud detection. *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- Xu, L., Skoularidou, M., Cuesta-Infante, A., and Veeramachaneni, K. Modeling tabular data using conditional gan. In *Advances in Neural Information Processing Systems*, 2019.
- Zhao, X., Guan, S., Xue, Y., and Pan, H. Hs-cgk: A hybrid sampling method for imbalance data based on conditional tabular generative adversarial network and k-nearest neighbor algorithm. *Computing and Informatics*, 43(1):213–239, 2024.
- Zhu, B., Pan, X., vanden Broucke, S., and Xiao, J. A gan-based hybrid sampling method for imbalanced customer classification. *Information Sciences*, 609:1397–1411, 2022.

A. Correlation Heat Map

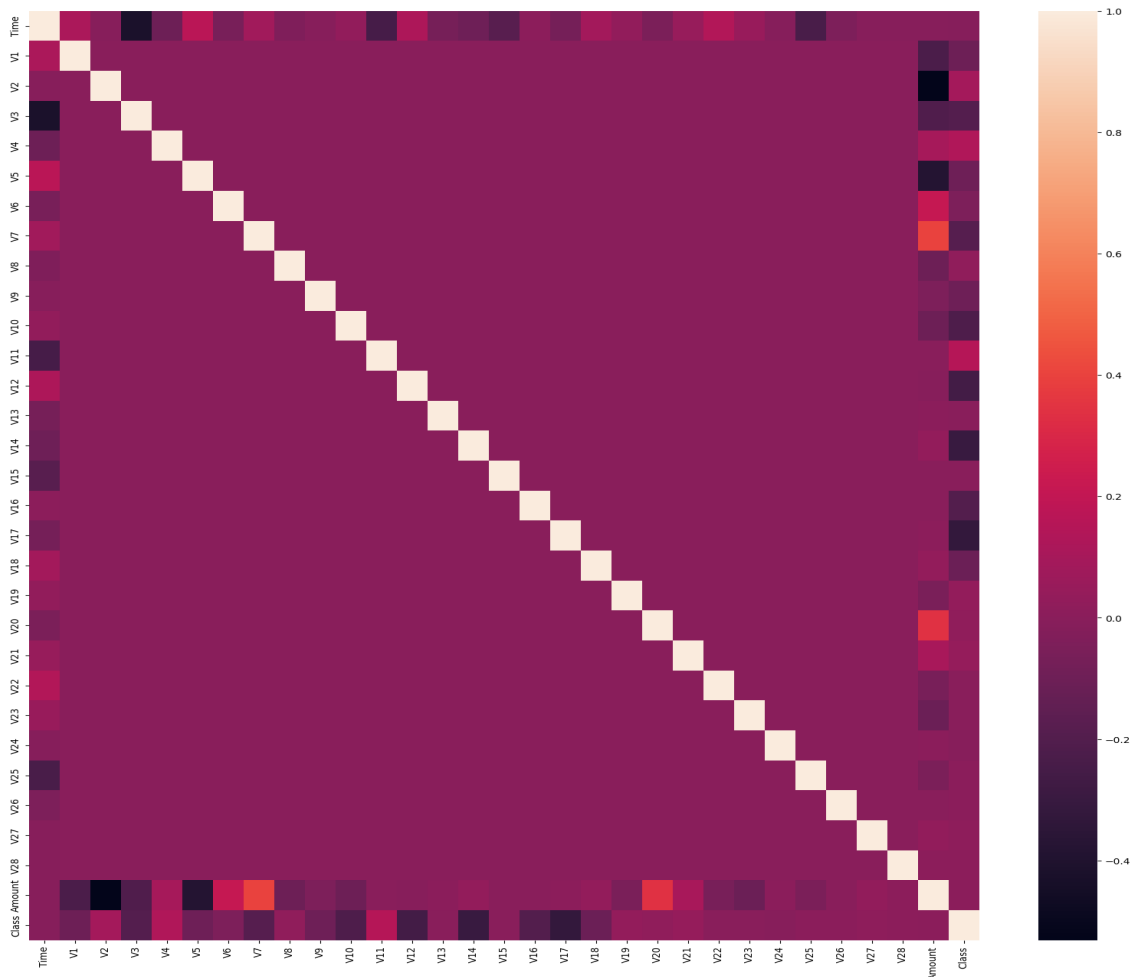


Figure 4. Correlation heat map for features in the credit card fraud dataset.

B. GAN Convergence Plots

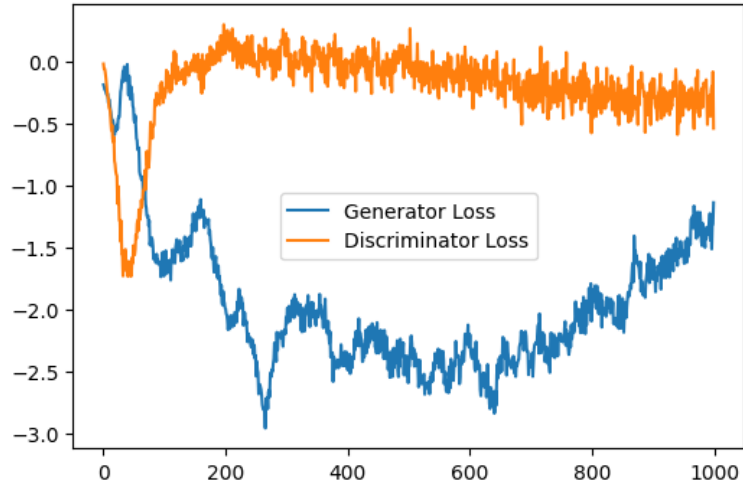


Figure 5. Convergence of CTGAN Model

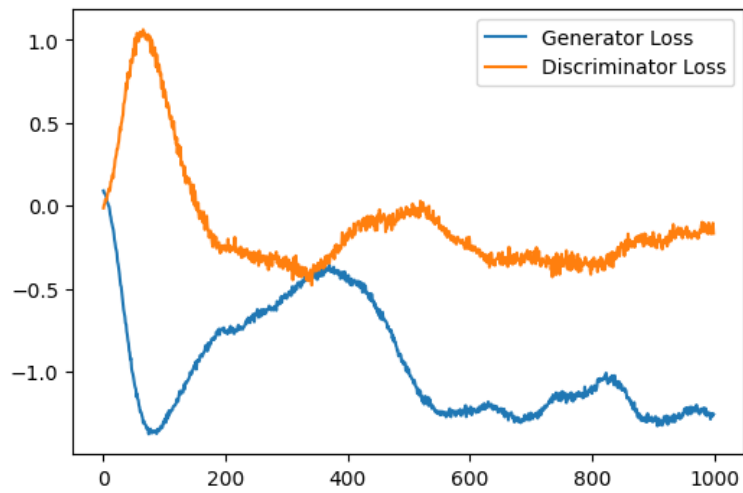


Figure 6. Convergence of DB-CTGAN-ENN Model

C. Ordered k-Nearest Neighbor Distance Plot

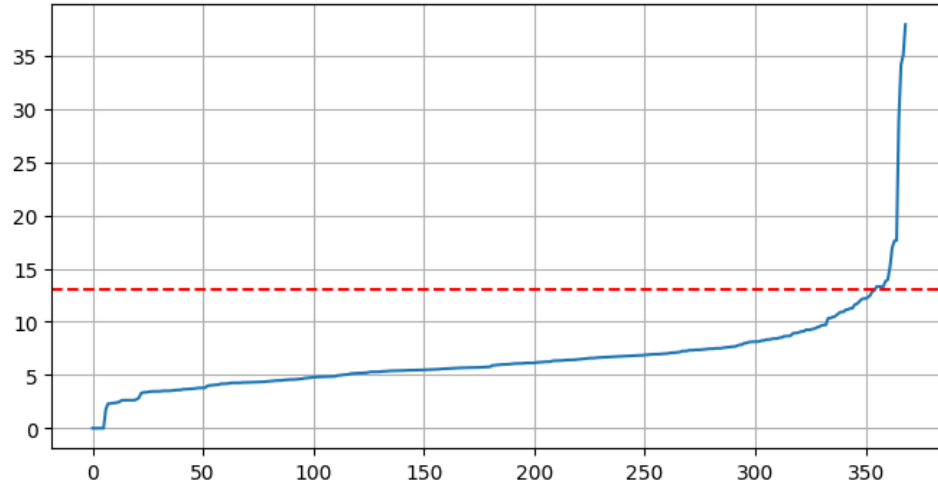


Figure 7. Ordered k-nearest neighbor distance plot of the empirical fraud samples with horizontal red line at 13.

D. Classifier performance metrics on the train set*Table 3.* Classifier performance metrics on the train set

CLASSIFIER	SAMPLING METHOD	RECALL	PRECISION	F-1	AUPRC
LOGISTIC REGRESSION	RUS	0.8238	0.6681	0.7379	0.7590
	SMOTE	0.8482	0.6161	0.7138	0.7691
	UNIVARIATE KDE	0.8509	0.5913	0.6978	0.7649
	CTGAN	0.7751	0.5575	0.6485	0.7028
	DB-CTGAN-ENN	0.8076	0.8301	0.8187	0.7639
MLP	RUS	0.8428	0.4677	0.6015	0.7607
	SMOTE	0.9675	0.6774	0.7969	0.8684
	UNIVARIATE KDE	0.9133	0.6713	0.7738	0.8419
	CTGAN	0.8157	0.7818	0.7984	0.7671
	DB-CTGAN-ENN	0.8103	0.8306	0.8203	0.7675
RANDOM FOREST	RUS	0.9268	0.6759	0.7817	0.9205
	SMOTE	1.0000	0.9736	0.9866	1.0000
	UNIVARIATE KDE	0.9566	0.9698	0.9632	0.9922
	CTGAN	0.8482	0.9601	0.9007	0.8806
	DB-CTGAN-ENN	0.8482	0.8485	0.8956	0.9555